

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240217 123240224
NAMA	:	Bisma Putra Pangestu Muhammad Syawal Azzami
PLUG	:	IF-E
NAMA ASISTEN	:	Khatama Putra Bintoro

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Bisma Putra Pangestu	123240217
Muhammad Syawal Azzami	123240224

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Asisten Praktikum

Khatama Putra
NIM. 123230053

Bintoro
NIM. 123230059

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul SPK Rekomendasi Tempat Wisata Indonesia Menggunakan SAW. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 25 Mei 2026

Penyusun 1,

Penyusun 2,

Bisma Putra Pangestu

NIM. 123240217

Muhammad Syawal Azzami

NIM. 123240224

DAFTAR ISI

HALAMAN PENGESAHAN	ii
KATA PENGANTAR.....	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Tujuan Proyek Akhir	1
1.3 Manfaat Proyek Akhir	2
BAB II PEMBAHASAN.....	3
2.1 Dasar Teori	3
2.2 Skenario Kasus & Dataset	4
2.3 Pemodelan Sistem Pendukung Keputusan	5
2.4 Perhitungan & Algoritma	6
2.5 Implementasi & Tampilan Sistem	7
BAB III JADWAL Pengerjaan dan Pembagian Tugas	27
3.1 Jadwal Pengerjaan	27
3.2 Pembagian Tugas.....	27
BAB IV KESIMPULAN DAN SARAN.....	28
4.2 Kesimpulan.....	28
4.3 Saran	28
DAFTAR PUSTAKA.....	29

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan sektor pariwisata di Indonesia semakin meningkat seiring dengan banyaknya destinasi wisata yang tersedia di berbagai kota. Banyaknya pilihan tempat wisata sering kali membuat wisatawan mengalami kesulitan dalam menentukan destinasi yang paling sesuai dengan kebutuhan dan preferensi mereka. Faktor seperti *rating* tempat wisata, harga tiket masuk, durasi kunjungan, kategori wisata, serta popularitas kota menjadi pertimbangan penting dalam pengambilan keputusan.

Proses pemilihan tempat wisata secara manual memerlukan waktu yang cukup lama karena pengguna harus membandingkan berbagai alternatif berdasarkan banyak kriteria. Oleh karena itu, diperlukan sebuah Sistem Pendukung Keputusan (SPK) yang dapat membantu pengguna dalam menentukan rekomendasi tempat wisata terbaik secara lebih cepat, objektif, dan efisien.

Pada proyek akhir ini digunakan metode *Simple Additive Weighting* (SAW) untuk melakukan proses perankingan tempat wisata berdasarkan bobot kriteria yang diberikan pengguna. Metode SAW dipilih karena memiliki konsep perhitungan yang sederhana, mudah dipahami, dan mampu menghasilkan rekomendasi terbaik berdasarkan nilai preferensi tertinggi.

Sistem ini dibangun menggunakan bahasa pemrograman *Python* dengan *framework Streamlit* sehingga dapat menampilkan hasil rekomendasi secara interaktif dan mudah digunakan oleh pengguna.

1.2 Tujuan Proyek Akhir

Tujuan dari proyek akhir ini adalah:

1. Membuat Sistem Pendukung Keputusan rekomendasi tempat wisata di Indonesia.
2. Mengimplementasikan metode *Simple Additive Weighting* (SAW) pada proses perankingan destinasi wisata.
3. Membantu pengguna dalam menentukan tempat wisata terbaik berdasarkan beberapa kriteria.
4. Menampilkan hasil rekomendasi wisata secara interaktif menggunakan *Streamlit*.

1.3 Manfaat Proyek Akhir

Manfaat dari proyek akhir ini adalah:

1. Membantu pengguna memilih tempat wisata sesuai preferensi dan kebutuhan.
2. Mempermudah proses pengambilan keputusan dengan mempertimbangkan banyak kriteria sekaligus.
3. Memberikan rekomendasi wisata secara objektif berdasarkan metode SAW.
4. Menjadi sarana pembelajaran implementasi Sistem Pendukung Keputusan menggunakan *Python* dan *Streamlit*.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Sistem Pendukung Keputusan (SPK)

Sistem Pendukung Keputusan atau *Decision Support System* (DSS) merupakan sistem berbasis komputer yang digunakan untuk membantu proses pengambilan keputusan dalam menyelesaikan suatu masalah semi terstruktur maupun tidak terstruktur. SPK membantu pengguna dalam menentukan pilihan terbaik berdasarkan data, model, dan metode tertentu.

Pada proyek ini, SPK digunakan untuk memberikan rekomendasi tempat wisata terbaik berdasarkan beberapa kriteria penilaian.

Metode *Simple Additive Weighting* (SAW)

Metode *Simple Additive Weighting* (SAW) merupakan salah satu metode dalam Sistem Pendukung Keputusan yang sering digunakan untuk proses perbandingan alternatif berdasarkan banyak kriteria. Metode SAW dikenal juga sebagai metode penjumlahan terbobot.

Konsep dasar metode SAW adalah mencari jumlah nilai terbobot dari setiap alternatif pada seluruh kriteria. Sebelum dilakukan perhitungan akhir, setiap nilai kriteria akan dinormalisasi terlebih dahulu agar memiliki skala yang sama.

Rumus normalisasi untuk kriteria benefit:

$$r_{ij} = \frac{x_{ij}}{\max(x_{ij})}$$

Keterangan:

- r_{ij} = nilai normalisasi
- x_{ij} = nilai asli alternatif
- $\max(x_{ij})$ = nilai maksimum pada kriteria

Rumus normalisasi untuk kriteria cost:

$$r_{ij} = \frac{\min(x_{ij})}{x_{ij}}$$

Rumus perhitungan nilai preferensi SAW:

$$V_i = \sum_{j=1}^n w_j r_{ij}$$

Keterangan:

- V_i = nilai akhir alternatif
- w_j = bobot kriteria
- r_{ij} = nilai normalisasi

Alternatif dengan nilai V_i terbesar akan menjadi rekomendasi terbaik.

2.2 Skenario Kasus & Dataset

Pada proyek akhir ini dibuat sebuah Sistem Pendukung Keputusan untuk merekomendasikan tempat wisata terbaik di Indonesia. Sistem ditujukan untuk membantu pengguna memilih destinasi wisata berdasarkan preferensi tertentu.

Permasalahan yang sering terjadi adalah pengguna kesulitan memilih tempat wisata karena banyaknya alternatif yang tersedia. Oleh karena itu, sistem ini membantu melakukan proses seleksi secara otomatis menggunakan metode SAW.

Pengambil Keputusan

Pengambil keputusan pada sistem ini adalah pengguna aplikasi yang ingin mencari rekomendasi tempat wisata terbaik.

Tujuan Sistem

Tujuan utama sistem adalah memberikan rekomendasi tempat wisata berdasarkan:

- Rating tempat wisata
- Harga tiket
- Durasi kunjungan
- Preferensi kategori wisata
- Popularitas kota

Dataset

Dataset yang digunakan bernama *tourism_with_id.csv* yang berisi data destinasi wisata di Indonesia.

Isi dataset meliputi:

- ID tempat wisata
- Nama tempat wisata
- Kota
- Kategori wisata
- Harga tiket

- Rating
- Durasi kunjungan

Jumlah data:

- 437 destinasi wisata

Dataset diperoleh dari:

- *Kaggle Indonesia Tourism Destination Dataset*

2.3 Pemodelan Sistem Pendukung Keputusan

Alternatif

Alternatif pada sistem ini adalah seluruh tempat wisata yang terdapat pada *dataset*.

Contoh alternatif:

1. Candi Borobudur
2. Taman Mini Indonesia Indah
3. Pantai Parangtritis
4. Museum Angkut
5. Jatim Park

Kriteria dan Tipe

Kode	Kriteria	Tipe
C1	<i>Rating</i>	<i>Benefit</i>
C2	Harga Tiket	<i>Cost</i>
C3	Durasi Kunjungan	<i>Benefit</i>
C4	Preferensi Kategori	<i>Benefit</i>
C5	Popularitas Kota	<i>Benefit</i>

Penjelasan Kriteria

1. *Rating*

Menunjukkan tingkat kepuasan pengunjung terhadap tempat wisata. Semakin tinggi *rating* maka semakin baik.

2. Harga Tiket

Menunjukkan biaya masuk tempat wisata. Semakin murah harga tiket maka semakin baik.

3. Durasi Kunjungan

Menunjukkan estimasi waktu kunjungan wisata. Semakin lama durasi dianggap semakin menarik untuk dikunjungi.

4. Preferensi Kategori

Skor kategori berdasarkan pilihan favorit pengguna seperti:

- Budaya
- Bahari
- Taman Hiburan
- Cagar Alam
- Tempat Ibadah
- Pusat Perbelanjaan

5. Popularitas Kota

Menunjukkan jumlah destinasi wisata pada suatu kota. Semakin populer kota maka nilainya semakin tinggi.

Bobot Kriteria

Bobot kriteria dapat diatur langsung oleh pengguna menggunakan *slider* pada aplikasi *Streamlit*.

Contoh bobot:

Kriteria	Bobot
<i>Rating</i>	5
Harga Tiket	4
Durasi	3
Preferensi Kategori	4
Popularitas Kota	2

Bobot kemudian dinormalisasi sebelum proses perhitungan SAW dilakukan.

2.4 Perhitungan & Algoritma

Langkah-langkah metode SAW pada sistem ini adalah:

1. Menentukan Kriteria dan Bobot

Pengguna menentukan tingkat kepentingan setiap kriteria menggunakan *slider*.

2. Membentuk Matriks Keputusan

Sistem mengambil nilai setiap alternatif berdasarkan seluruh kriteria.

3. Normalisasi Matriks

Dilakukan normalisasi sesuai tipe kriteria:

- *Benefit* menggunakan nilai maksimum

- *Cost* menggunakan nilai minimum

4. Menghitung Nilai Preferensi

Setiap nilai normalisasi dikalikan dengan bobot kriteria.

5. Proses Perangkingan

Nilai preferensi terbesar menjadi rekomendasi terbaik.

Algoritma Sistem

Berikut alur algoritma sistem:

1. Membaca *dataset CSV*
2. Melakukan *preprocessing data*
3. Menentukan bobot dan preferensi pengguna
4. Melakukan normalisasi SAW
5. Menghitung nilai preferensi
6. Mengurutkan alternatif berdasarkan nilai terbesar
7. Menampilkan hasil rekomendasi

2.5 Implementasi & Tampilan Sistem

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns

# KONFIGURASI HALAMAN
st.set_page_config(
    page_title="SPK Wisata Indonesia - SAW",
    page_icon="",
    layout="wide",
)

# WARNA TEMA
PALETTE = ["#00B3FF", "#FBFF00", "#FF9900", "#FF3300", "#FF006F",
"#00FF3C"]
sns.set_theme(style="whitegrid", palette=PALETTE)

# FUNGSI BANTU
@st.cache_data
def load_data(path: str) -> pd.DataFrame:
    """Muat dataset dan lakukan preprocessing awal."""
    df = pd.read_csv(path)

    # Buang kolom kosong / tidak relevan
    df = df.drop(columns=["Unnamed: 11", "Unnamed: 12"], errors="ignore")

    # Isi Time_Minutes yang kosong dengan median (60 menit)
```

```

df["Time_Minutes"] = df["Time_Minutes"].fillna(df["Time_Minutes"].median())

# — Kriteria 4 : Skor Kategori (1-6, default merata)
cat_rank = {
    "Taman Hiburan": 6,
    "Budaya": 5,
    "Cagar Alam": 4,
    "Bahari": 3,
    "Tempat Ibadah": 2,
    "Pusat Perbelanjaan": 1,
}
df["Category_Score"] = df["Category"].map(cat_rank).fillna(1)

# — Kriteria 5 : Skor Popularitas Kota (jumlah destinasi per kota)
city_pop = df["City"].value_counts().to_dict()
df["City_Popularity"] = df["City"].map(city_pop)

return df

def build_category_score(df: pd.DataFrame, fav_category: str) -> pd.Series:
    """
    Hitung ulang Category_Score berdasarkan pilihan user.
    Kategori favorit user = 6, sisanya turun berurutan.
    """
    all_cats = [
        "Taman Hiburan", "Budaya", "Cagar Alam",
        "Bahari", "Tempat Ibadah", "Pusat Perbelanjaan",
    ]
    # Rotasi urutan sehingga fav_category berada di posisi pertama (nilai 6)
    reordered = [fav_category] + [c for c in all_cats if c != fav_category]
    rank_map = {cat: 6 - i for i, cat in enumerate(reordered)}
    return df["Category"].map(rank_map).fillna(1)

def normalize_saw(df: pd.DataFrame, criteria: dict) -> pd.DataFrame:
    """
    Normalisasi matriks keputusan menggunakan rumus SAW dengan NumPy.

    Benefit :  $r_{ij} = x_{ij} / \max(x_j)$  → np.max()
    Cost :  $r_{ij} = \min(x_j) / x_{ij}$  → np.min(), np.where()
           Khusus jika min = 0 (misal harga gratis):
            $r_{ij} = 1.0$  jika  $x_{ij} == 0$  (gratis = terbaik)
            $r_{ij} = \min\_nonzero / x_{ij}$  untuk nilai > 0
    """
    norm = pd.DataFrame(index=df.index)
    for col, ctype in criteria.items():
        arr = df[col].to_numpy(dtype=float) # ← konversi ke NumPy array

        if ctype == "benefit":
            col_max = np.max(arr) # ← np.max()
            norm[col] = np.where( # ← np.where()
                col_max == 0,
                0.0,
                arr / col_max
            )

```

```

        else: # cost
            col_min = np.min(arr) # ← np.min()
            if col_min == 0:
                # Ambil nilai minimum yang bukan nol menggunakan np.where
                nonzero_vals = arr[np.where(arr > 0)] # ← np.where
                min_nonzero = np.min(nonzero_vals) if len(nonzero_vals)
> 0 else 1.0
                # np.where: jika 0 → 1.0 (gratis=sempurna), jika >0 →
                min/x
            with np.errstate(divide="ignore", invalid="ignore"): #
← np.errstate()
                norm[col] = np.where(
                    arr == 0,
                    1.0,
                    min_nonzero / arr
                )
            else:
                norm[col] = col_min / arr # ← operasi vektor
NumPy

    return norm

def compute_saw(df: pd.DataFrame, bobot: dict, fav_cat: str) ->
pd.DataFrame:
    """
    Hitung nilai SAW menggunakan operasi matriks NumPy.
 $V_i = \sum w_j * r_{ij} \rightarrow \text{np.dot}(\text{matriks\_norm}, \text{vektor\_bobot})$ 
    """
    data = df.copy()

    # Update Category_Score berdasarkan pilihan user
    data["Category_Score"] = build_category_score(data, fav_cat)

    criteria_cols = {
        "Rating": "benefit",
        "Price": "cost",
        "Time_Minutes": "benefit",
        "Category_Score": "benefit",
        "City_Popularity": "benefit",
    }

    # Normalisasi → hasilnya DataFrame
    norm_df = normalize_saw(data, criteria_cols)

    # Bobot dinormalisasi menggunakan NumPy
    bobot_arr = np.array([bobot[k] for k in criteria_cols],
dtype=float) # ← np.array()
    bobot_norm = bobot_arr /
np.sum(bobot_arr) # ← np.sum()
    norm_weights = dict(zip(criteria_cols.keys(), bobot_norm))

    # Hitung  $V_i = \text{dot product matriks normalisasi} \times \text{vektor bobot}$ 
    # np.dot() jauh lebih efisien daripada loop manual
    matriks_norm =
norm_df[list(criteria_cols.keys())].to_numpy(dtype=float) # ← np.array
2D

```

```

                                nilai_saw_arr      =      np.dot(matriks_norm,
bobot_norm)                                # ← np.dot()

    # Ganti NaN/inf dengan 0 menggunakan np.nan_to_num
    nilai_saw_arr      =      np.nan_to_num(nilai_saw_arr,      nan=0.0,
posinf=0.0)      # ← np.nan_to_num()

    data["Nilai_SAW"] = nilai_saw_arr

    # Peringkat menggunakan np.argsort
    # argsort descending → urutan indeks dari nilai terbesar ke terkecil
                                urutan      =      np.argsort(-
nilai_saw_arr)                                # ← np.argsort()
    peringkat = np.empty_like(urutan)
    peringkat[urutan]      =      np.arange(1,      len(urutan)      +
1)      # ← np.arange()
    data["Peringkat"] = peringkat

    data = data.sort_values("Peringkat")
    return data, norm_df, norm_weights

# MUAT DATA
DATA_PATH = "tourism_with_id.csv"

try:
    df_raw = load_data(DATA_PATH)
except FileNotFoundError:
    st.error(
        "File `tourism_with_id.csv` tidak ditemukan. "
        "Pastikan file berada di folder yang sama dengan script ini."
    )
    st.stop()

# SIDEBAR — NAVIGASI & INPUT BOBOT
with st.sidebar:
    st.image(
        "https://kab-jayawijaya.kpu.go.id/public/kab-
jayawijaya/images/1760966694_dc3d3c79c4bfdc81974e.jpg",
        width=250,
    )
    st.title("SPK Wisata Indonesia")
    st.divider()

    halaman = st.radio(
        "Navigasi",
        ["Beranda", "Dataset", "Hitung SPK", "Visualisasi", "Profil
Kelompok"],
        index=0,
    )
    st.divider()

    # Input Bobot (widget interaktif)
    st.subheader("Bobot Kriteria")
    st.caption("Geser slider untuk mengatur bobot tiap kriteria.")

    w_rating      =      st.slider("Rating",                                min_value=1,
max_value=10, value=5)
    w_price      =      st.slider("Harga Tiket",                                min_value=1,
max_value=10, value=4)

```

```

w_time = st.slider("Durasi Kunjungan", min_value=1,
max_value=10, value=3)
w_category = st.slider("Preferensi Kategori", min_value=1,
max_value=10, value=4)
w_city = st.slider("Popularitas Kota", min_value=1,
max_value=10, value=2)

bobot = {
    "Rating": w_rating,
    "Price": w_price,
    "Time_Minutes": w_time,
    "Category_Score": w_category,
    "City_Popularity": w_city,
}

total_w = sum(bobot.values())
st.caption(f"Total bobot saat ini: **{total_w}** (akan dinormalisasi
otomatis)")
st.divider()

# Preferensi Kategori (Selectbox)
st.subheader("Preferensi Kategori")
fav_category = st.selectbox(
    "Pilih kategori wisata favorit Anda:",
    options=[
        "Taman Hiburan", "Budaya", "Cagar Alam",
        "Bahari", "Tempat Ibadah", "Pusat Perbelanjaan",
    ],
    index=0,
)

#Filter Kota (Multiselect)
st.subheader("Filter Kota")
all_cities = sorted(df_raw["City"].unique().tolist())
selected_cities = st.multiselect(
    "Tampilkan kota:",
    options=all_cities,
    default=all_cities,
)
if not selected_cities:
    selected_cities = all_cities # default semua

st.divider()

# FILTER DATA BERDASARKAN KOTA
df_filtered = df_raw[df_raw["City"].isin(selected_cities)].copy()

# HALAMAN : BERANDA
if halaman == "Beranda":
    st.title("Sistem Pendukung Keputusan")
    st.subheader("Rekomendasi Tempat Wisata Terbaik di Indonesia")
    st.markdown(
        """
        Aplikasi ini membantu Anda menemukan tempat wisata terbaik
menggunakan
        metode **Simple Additive Weighting (SAW)**.

        ### Tentang Dataset

```

Dataset berisi informasi **437 destinasi wisata** di 5 kota besar Indonesia.

```

### 5 Kriteria Penilaian
| No | Kriteria | Tipe | Keterangan |
|:--:|:-----:|:----:|:-----:|
| 1 | Rating | Benefit | Penilaian rata-rata dari pengunjung (3.4 - 5.0) |
| 2 | Harga Tiket | Cost | Tiket masuk dalam Rupiah - semakin murah = lebih baik |
| 3 | Durasi Kunjungan | Benefit | Estimasi waktu kunjungan (menit) |
| 4 | Preferensi Kategori | Benefit | Skor berdasarkan kategori favorit Anda |
| 5 | Popularitas Kota | Benefit | Jumlah destinasi di kota tersebut |

```

```

### Rumus SAW
"""
)
st.latex(r"V_i = \sum_{j=1}^n w_j \cdot r_{ij}")
st.markdown(
    """
    Di mana:
    - $V_i$ = Nilai preferensi alternatif ke-$i$
    - $w_j$ = Bobot kriteria ke-$j$ (sudah dinormalisasi)
    - $r_{ij}$ = Nilai ternormalisasi alternatif $i$ pada kriteria $j$
    """
)

```

```

Normalisasi :
"""
)
col1, col2 = st.columns(2)
with col1:
    st.markdown("Kriteria Benefit")
    st.latex(r"r_{ij} = \frac{x_{ij}}{\max_i(x_{ij})}")
with col2:
    st.markdown("Kriteria Cost")
    st.latex(r"r_{ij} = \frac{\min_i(x_{ij})}{x_{ij}}")

st.divider()

```

```

# Statistik ringkas
col1, col2, col3, col4 = st.columns(4)
col1.metric("Total Destinasi", f"{len(df_raw):,}")
col2.metric("Kota", df_raw["City"].nunique())
col3.metric("Kategori", df_raw["Category"].nunique())
col4.metric("Rata-rata Rating", f"{df_raw['Rating'].mean():.2f}")

```

```

# HALAMAN : DATASET
elif halaman == "Dataset":
    st.title("Dataset Tempat Wisata Indonesia")

    st.info(
        f"Menampilkan {len(df_filtered):,} baris data "
        f"dari kota: {'', '.join(selected_cities)}"
    )

    # Tabel interaktif

```



```

tampil_cols = [
    "Place_Id", "Place_Name", "Category", "City",
    "Price", "Rating", "Time_Minutes",
]
st.dataframe(
    df_filtered[tampil_cols].rename(columns={
        "Place_Id": "ID",
        "Place_Name": "Nama Tempat",
        "Category": "Kategori",
        "City": "Kota",
        "Price": "Harga (Rp)",
        "Rating": "Rating",
        "Time_Minutes": "Durasi (menit)",
    }),
    use_container_width=True,
    height=450,
)

st.divider()

# HALAMAN : HITUNG SPK
elif halaman == "Hitung SPK":
    st.title("Perhitungan SPK - Metode SAW")

    # Tampilkan bobot yang dipilih
    st.subheader("Konfigurasi Bobot")
    total_w = sum(bobot.values())
    norm_w = {k: round(v / total_w, 4) for k, v in bobot.items()}

    bobot_df = pd.DataFrame({
        "Kriteria": ["Rating", "Harga Tiket", "Durasi Kunjungan", "Pref.
Kategori", "Popularitas Kota"],
        "Tipe": ["Benefit", "Cost", "Benefit", "Benefit", "Benefit"],
        "Bobot Asli": list(bobot.values()),
        "Bobot Norm.": list(norm_w.values()),
    })
    st.dataframe(bobot_df, use_container_width=True, hide_index=True)

    st.markdown(f"Kategori Favorit Anda: {fav_category}")
    st.divider()

    # Tombol eksekusi
    if st.button("Jalankan Perhitungan SAW", type="primary",
use_container_width=True):
        if df_filtered.empty:
            st.warning("Tidak ada data untuk dihitung. Periksa filter
kota Anda.")
            st.stop()

        with st.spinner("Sedang menghitung nilai SAW..."):
            df_result, norm_df, norm_w_used = compute_saw(df_filtered,
bobot, fav_category)

            st.success("Perhitungan selesai!")
            st.divider()

    # — Step 1 : Matriks Keputusan Awal —
    st.subheader("Langkah 1 - Matriks Keputusan (10 Teratas)")

```

```

        st.caption("Nilai asli dari tiap alternatif untuk setiap
kriteria.")
        raw_cols = ["Place_Name", "Rating", "Price", "Time_Minutes",
"Category_Score", "City_Popularity"]

        matriks_df = df_result[raw_cols].head(10).copy()

        # Format kolom Harga: tampilkan "Gratis" jika 0, "Rp X,XXX" jika
> 0
        matriks_df["Price"] = matriks_df["Price"].apply(
            lambda x: "Gratis" if x == 0 else f"Rp {int(x):,}")
        )
        # Format durasi: tampilkan sebagai "X menit"
        matriks_df["Time_Minutes"] = matriks_df["Time_Minutes"].apply(
            lambda x: f"{int(x)} menit"
        )

        st.dataframe(
            matriks_df.rename(columns={
                "Place_Name": "Nama Tempat",
                "Rating": "Rating",
                "Price": "Harga Tiket",
                "Time_Minutes": "Durasi",
                "Category_Score": "Skor Kategori",
                "City_Popularity": "Popularitas Kota",
            }),
            use_container_width=True,
            hide_index=True,
        )

        st.caption("***Catatan:** *Gratis* = harga tiket Rp 0 (masuk
gratis). "
                    "Pada normalisasi SAW, harga Rp 0 mendapat nilai
**1.0** (terbaik) "
                    "karena kriteria Harga bersifat Cost – semakin murah
semakin baik.")

        # Step 2 : Matriks Ternormalisasi
        st.subheader("Langkah 2 - Matriks Normalisasi (10 Teratas)")
        st.caption("Nilai setiap kriteria setelah dinormalisasi dengan
rumus SAW.")
        norm_show = norm_df.loc[df_result.index].head(10).copy()
        norm_show.insert(0, "Nama Tempat",
df_result["Place_Name"].values[:10])
        norm_show = norm_show.rename(columns={
            "Rating": "Rating (norm)",
            "Price": "Harga (norm)",
            "Time_Minutes": "Durasi (norm)",
            "Category_Score": "Kategori (norm)",
            "City_Popularity": "Popularitas (norm)",
        })
        st.dataframe(norm_show.round(4), use_container_width=True,
hide_index=True)

        # Step 3 : Nilai SAW & Bobot
        st.subheader("Langkah 3 - Bobot Ternormalisasi yang Digunakan")
        w_df = pd.DataFrame({
            "Kriteria": ["Rating", "Harga Tiket", "Durasi", "Pref.
Kategori", "Popularitas Kota"],
            "Bobot (w)": [round(norm_w_used[k], 4) for k in

```

```

["Rating", "Price", "Time_Minutes",
"Category_Score", "City_Popularity"]],
    })
    st.dataframe(w_df, use_container_width=True, hide_index=True)

    # — Step 4 : Tabel Hasil Perangkingan
    st.subheader("Langkah 4 - Hasil Perangkingan Akhir")
    st.caption("Diurutkan dari nilai tertinggi (Peringkat 1) hingga
    terendah.")

    result_cols = [
        "Peringkat", "Place_Name", "Category", "City",
        "Price", "Rating", "Time_Minutes", "Nilai_SAW",
    ]
    result_display = df_result[result_cols].rename(columns={
        "Peringkat": "Peringkat",
        "Place_Name": "Nama Tempat",
        "Category": "Kategori",
        "City": "Kota",
        "Price": "Harga (Rp)",
        "Rating": "Rating",
        "Time_Minutes": "Durasi (menit)",
        "Nilai_SAW": "Nilai SAW",
    })
    result_display["Nilai SAW"] = result_display["Nilai
    SAW"].round(4)

    st.dataframe(result_display, use_container_width=True,
    hide_index=True, height=500)

    # Simpan ke session state untuk halaman visualisasi
    st.session_state["df_result"] = df_result

    # Top 5 sorotan
    st.divider()
    st.subheader("Top 5 Rekomendasi Terbaik")
    top5 = df_result.head(5)
    cols = st.columns(5)
    for i, (_, row) in enumerate(top5.iterrows()):
        with cols[i]:
            st.markdown(f"**{row['Place_Name']}**")
            st.markdown(f"{row['City']}")
            st.markdown(f"{row['Category']}")
            st.markdown(f"{row['Rating']}")
            st.markdown(f"Rp {int(row['Price']):,}")
            st.markdown(f"SAW: `{row['Nilai_SAW']:.4f}`")

    else:
        st.info("Atur bobot di sidebar, lalu tekan tombol **Jalankan
        Perhitungan SAW** untuk melihat hasil.")

    # HALAMAN : VISUALISASI
    elif halaman == "Visualisasi":
        st.title("Visualisasi Data Analitik")
        # Grafik 1 : plt.bar() - Jumlah Destinasi per Kota
        st.subheader("Grafik 1 - Jumlah Destinasi Wisata per Kota (bar)")
        fig1, ax1 = plt.subplots(figsize=(9, 5))
        city_count = df_filtered["City"].value_counts()
        bars1 = ax1.bar(

```

```

        city_count.index,
        city_count.values,
        color=PALETTE[:len(city_count)],
        edgecolor="white",
        linewidth=0.8,
        width=0.55,
    )
    # Label angka di atas tiap batang
    for bar in bars1:
        h = bar.get_height()
        ax1.text(
            bar.get_x() + bar.get_width() / 2, h + 1.5,
            str(int(h)), ha="center", va="bottom", fontsize=11,
            fontweight="bold"
        )
    ax1.set_xlabel("Kota", fontsize=11)
    ax1.set_ylabel("Jumlah Destinasi", fontsize=11)
    ax1.set_title("Jumlah Destinasi Wisata per Kota", fontsize=13,
        fontweight="bold")
    ax1.set_ylim(0, city_count.max() * 1.18)
    ax1.grid(axis="y", linestyle="--", alpha=0.4)
    fig1.tight_layout()
    st.pyplot(fig1)
    plt.close(fig1)

    st.divider()

    # Grafik 2 : plt.scatter() - Rating vs Harga per Kategori
    st.subheader("Grafik 2 - Hubungan Rating vs Harga Tiket per Kategori
(scatter)")
    fig2, ax2 = plt.subplots(figsize=(10, 6))
    categories = df_filtered["Category"].unique()
    for i, cat in enumerate(categories):
        subset = df_filtered[df_filtered["Category"] == cat]
        ax2.scatter(
            subset["Price"],
            subset["Rating"],
            label=cat,
            alpha=0.72,
            s=65,
            color=PALETTE[i % len(PALETTE)],
            edgecolors="white",
            linewidths=0.4,
        )
    ax2.set_xlabel("Harga Tiket (Rp)", fontsize=11)
    ax2.set_ylabel("Rating", fontsize=11)
    ax2.set_title("Hubungan Rating vs Harga Tiket per Kategori",
        fontsize=13, fontweight="bold")
    # Format sumbu X menjadi Rp
    ax2.xaxis.set_major_formatter(
        mticker.FuncFormatter(lambda x, _: f"Rp {int(x):,}")
    )
    plt.xticks(rotation=15, ha="right")
    ax2.legend(title="Kategori", fontsize=9, loc="lower right")
    ax2.grid(linestyle="--", alpha=0.3)
    fig2.tight_layout()
    st.pyplot(fig2)
    plt.close(fig2)

```

```

st.divider()

# Grafik 3 : plt.plot() - Rata-rata Rating per Kategori (Line)
st.subheader("Grafik 3 - Rata-rata Rating per Kategori Wisata
(plot)")
fig3, ax3 = plt.subplots(figsize=(9, 5))
avg_rating = (
    df_filtered.groupby("Category")["Rating"]
    .mean()
    .sort_values(ascending=False)
    .reset_index()
)
ax3.plot(
    avg_rating["Category"],
    avg_rating["Rating"],
    marker="o",
    markersize=9,
    linewidth=2.2,
    color=PALETTE[0],
    markerfacecolor=PALETTE[2],
    markeredgecolor="white",
    markeredgewidth=1.2,
)
# Anotasi nilai di tiap titik
for _, row in avg_rating.iterrows():
    ax3.annotate(
        f"{row['Rating']:.2f}",
        (row["Category"], row["Rating"]),
        textcoords="offset points",
        xytext=(0, 10),
        ha="center",
        fontsize=10,
        fontweight="bold",
        color=PALETTE[0],
    )
ax3.set_xlabel("Kategori", fontsize=11)
ax3.set_ylabel("Rata-rata Rating", fontsize=11)
ax3.set_title("Rata-rata Rating per Kategori Wisata", fontsize=13,
fontweight="bold")
ax3.set_ylim(avg_rating["Rating"].min() - 0.1,
avg_rating["Rating"].max() + 0.2)
plt.xticks(rotation=18, ha="right")
ax3.grid(linestyle="--", alpha=0.35)
fig3.tight_layout()
st.pyplot(fig3)
plt.close(fig3)

st.divider()

# Grafik 4 : plt.pie() - Proporsi Kategori Wisata
st.subheader("Grafik 4 - Proporsi Jumlah Destinasi per Kategori
(pie)")
fig4, ax4 = plt.subplots(figsize=(8, 6))
cat_count = df_filtered["Category"].value_counts()
wedges, texts, autotexts = ax4.pie(
    cat_count.values,
    labels=cat_count.index,
    autopct="%1.1f%%",
    colors=PALETTE[:len(cat_count)],

```

```

        startangle=140,
        pctdistance=0.78,
        wedgeprops={"edgecolor": "white", "linewidth": 1.5},
    )
    for at in autotexts:
        at.set_fontsize(9)
        at.set_fontweight("bold")
    ax4.set_title("Proporsi Jumlah Destinasi per Kategori", fontsize=13,
fontweight="bold")
    fig4.tight_layout()
    st.pyplot(fig4)
    plt.close(fig4)

    st.divider()

    # Grafik 5 : plt.bar() horizontal - Top 10 SAW (setelah dihitung)
    st.subheader("Grafik 5 - Top 10 Rekomendasi Nilai SAW (bar
horizontal)")
    if "df_result" in st.session_state:
        df_res = st.session_state["df_result"]
        top10 = df_res.head(10).copy()

        fig5, ax5 = plt.subplots(figsize=(10, 6))
        colors5 = [PALETTE[0]] + [PALETTE[1]] * (len(top10) - 1)
        bars5 = ax5.barh(
            top10["Place_Name"].values[::-1],
            top10["Nilai_SAW"].values[::-1],
            color=colors5[::-1],
            edgecolor="white",
            height=0.6,
        )
        # Label nilai di ujung batang
        for bar in bars5:
            w = bar.get_width()
            ax5.text(
                w + 0.002, bar.get_y() + bar.get_height() / 2,
                f"{w:.4f}", va="center", ha="left", fontsize=9
            )
        ax5.set_xlabel("Nilai SAW", fontsize=11)
        ax5.set_title("Top 10 Rekomendasi Destinasi Wisata (Nilai SAW)",
fontsize=13, fontweight="bold")
        ax5.set_xlim(0, top10["Nilai_SAW"].max() * 1.14)
        ax5.grid(axis="x", linestyle="--", alpha=0.35)
        fig5.tight_layout()
        st.pyplot(fig5)
        plt.close(fig5)
    else:
        st.info("Jalankan perhitungan SAW terlebih dahulu di halaman
Hitung SPK untuk menampilkan grafik ini.")

    # HALAMAN : PROFIL KELOMPOK
    elif halaman == "Profil Kelompok":
        st.title("Profil Kelompok")

        st.markdown(
            """
            ### Informasi Proyek
            | | |
            |--|--|

```

```

| **Mata Kuliah** | Sistem dan Cerdas Pendukung Keputusan (SCPK)
|
| **Tahun Ajaran** | 2025 / 2026 |
| **Metode SPK** | Simple Additive Weighting (SAW) |
| **Dataset** | Indonesia Tourism Destination (437 baris) |
| **Sumber Data** | Kaggle - Indonesia Tourism Destination |

---

### Anggota Kelompok
"""
)

coll1, col2 = st.columns(2)
with coll1:
    st.markdown(
        """
        **Anggota 1**
        - **Nama **: *Bisma Putra Pangestu*
        - **NIM **: *123240217*
        """
    )
with col2:
    st.markdown(
        """
        **Anggota 2**
        - **Nama **: *Muhammad Syawal Azzami*
        - **NIM **: *123240224*
        """
    )

st.divider()

```



[Deploy](#)

SPK Wisata Indonesia

Navigasi

- ☒ Beranda
- ☐ Dataset
- ☐ Hitung SPK
- ☐ Visualisasi
- ☐ Profil Kelompok

Bobot Kriteria

Geser slider untuk mengatur bobot tiap

Sistem Pendukung Keputusan

Rekomendasi Tempat Wisata Terbaik di Indonesia

Aplikasi ini membantu Anda menemukan tempat wisata terbaik menggunakan metode Simple Additive Weighting (SAW).

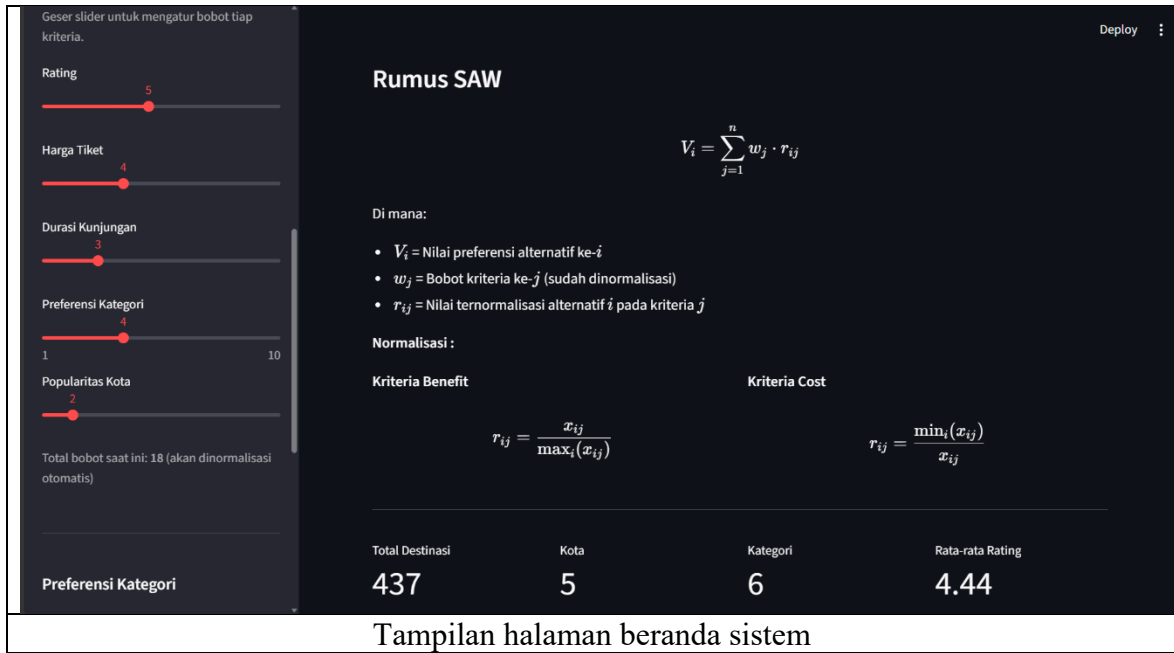
Tentang Dataset

Dataset berisi informasi 437 destinasi wisata di 5 kota besar Indonesia.

5 Kriteria Penilaian

No	Kriteria	Tipe	Keterangan
1	Rating	Benefit	Penilaian rata-rata dari pengunjung (3.4 - 5.0)
2	Harga Tiket	Cost	Tiket masuk dalam Rupiah - semakin murah = lebih baik
3	Durasi Kunjungan	Benefit	Estimasi waktu kunjungan (menit)
4	Preferensi Kategori	Benefit	Skor berdasarkan kategori favorit Anda

Tampilan halaman beranda sistem





SPK Wisata Indonesia

Navigasi
☐ Beranda
☒ Dataset
☐ Hitung SPK
☐ Visualisasi
☐ Profil Kelompok

Bobot Kriteria
 Geser slider untuk mengatur bobot tiap

Deploy

Dataset Tempat Wisata Indonesia

Menampilkan 437 baris data dari kota: Bandung, Jakarta, Semarang, Surabaya, Yogyakarta

	ID	Nama Tempat	Kategori	Kota	Harga (Rp)	Rating	Durasi (menit)
0	1	Monumen Nasional	Budaya	Jakarta	20000	4.6	15
1	2	Kota Tua	Budaya	Jakarta	0	4.6	90
2	3	Dunia Fantasi	Taman Hiburan	Jakarta	270000	4.6	360
3	4	Taman Mini Indonesia Indah (TMII)	Taman Hiburan	Jakarta	10000	4.5	60
4	5	Atlantis Water Adventure	Taman Hiburan	Jakarta	94000	4.5	60
5	6	Taman Impian Jaya Ancol	Taman Hiburan	Jakarta	25000	4.5	10
6	7	Kebun Binatang Ragunan	Cagar Alam	Jakarta	4000	4.5	60
7	8	Ocean Ecopark	Taman Hiburan	Jakarta	180000	4	60
8	9	Pelabuhan Marina	Bahari	Jakarta	175000	4.4	60
9	10	Pulau Tidung	Bahari	Jakarta	150000	4.5	60
10	11	Pulau Bidadari	Bahari	Jakarta	5000	4.6	60
11	12	Pulau Pari	Bahari	Jakarta	150000	4	60

Tampilan dataset wisata



SPK Wisata Indonesia

Navigasi
☐ Beranda
☐ Dataset
☒ Hitung SPK
☐ Visualisasi
☐ Profil Kelompok

Bobot Kriteria
 Geser slider untuk mengatur bobot tiap

Deploy

Perhitungan SPK – Metode SAW

Konfigurasi Bobot

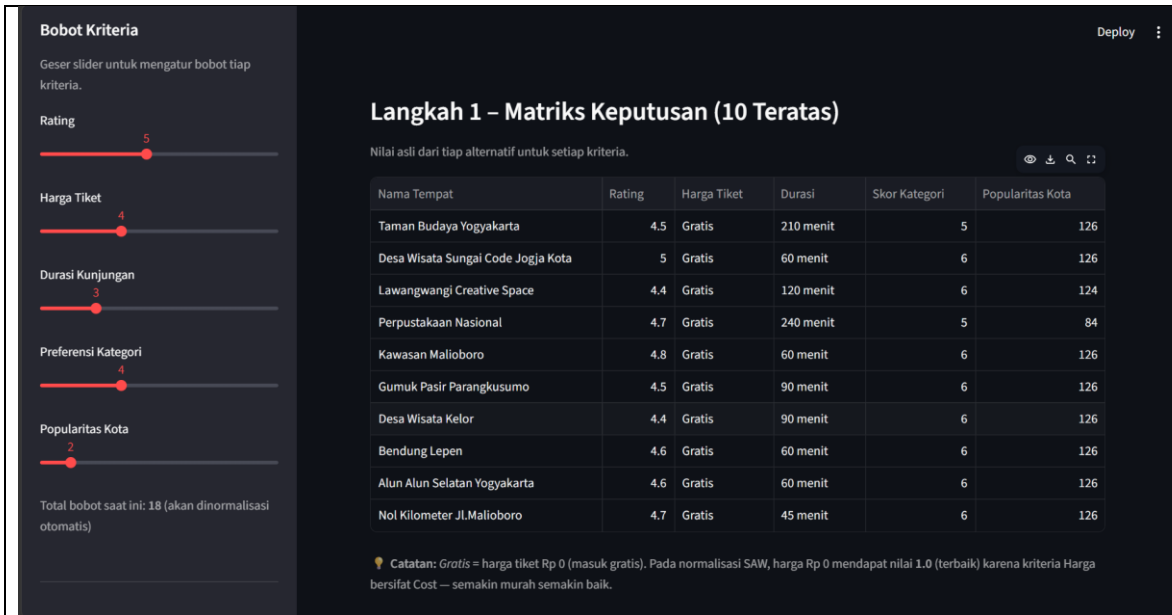
Kriteria	Tipe	Bobot Asli	Bobot Norm.
Rating	Benefit	5	0.2778
Harga Tiket	Cost	4	0.2222
Durasi Kunjungan	Benefit	3	0.1667
Pref. Kategori	Benefit	4	0.2222
Popularitas Kota	Benefit	2	0.1111

Kategori Favorit Anda: Taman Hiburan

Jalankan Perhitungan SAW

Atur bobot di sidebar, lalu tekan tombol Jalankan Perhitungan SAW untuk melihat hasil.

Tampilan proses perhitungan SAW Taman Hiburan



Tampilan proses perhitungan SAW pada langkah 1



Tampilan proses perhitungan SAW pada langkah 2



Tampilan proses perhitungan SAW pada langkah 3

Langkah 4 – Hasil Perangkingan Akhir

Diurutkan dari nilai tertinggi (Peringkat 1) hingga terendah.

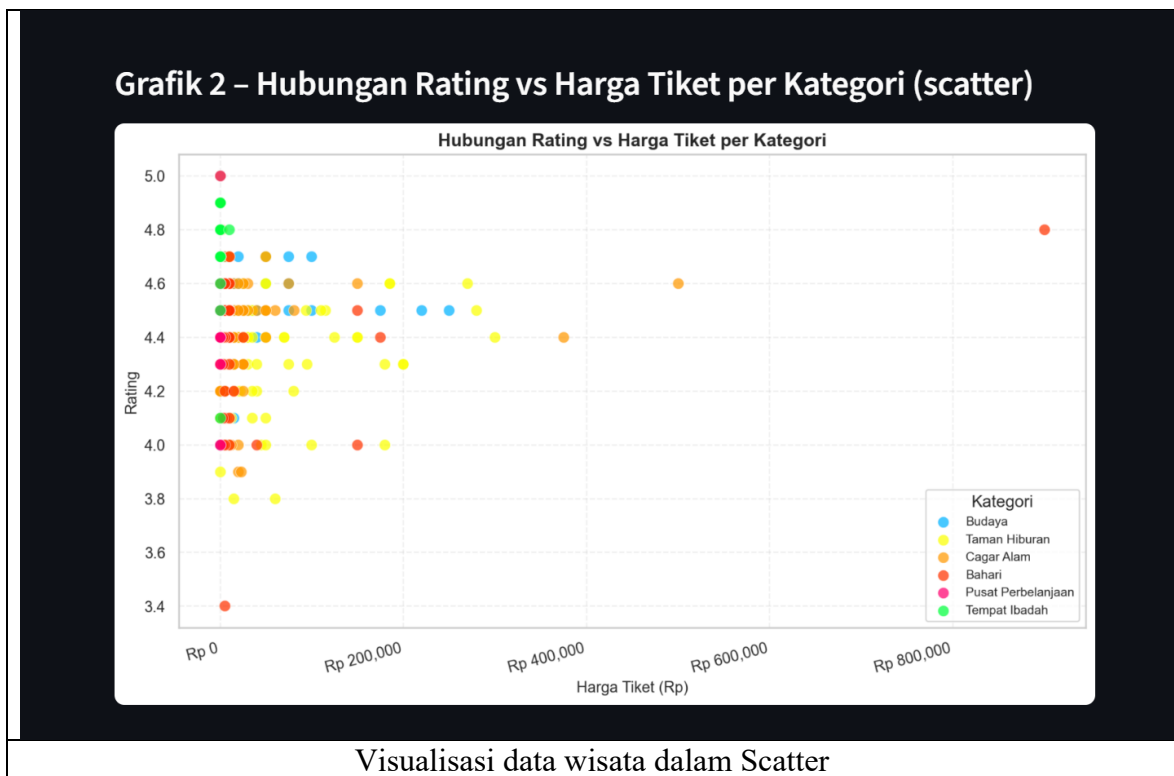
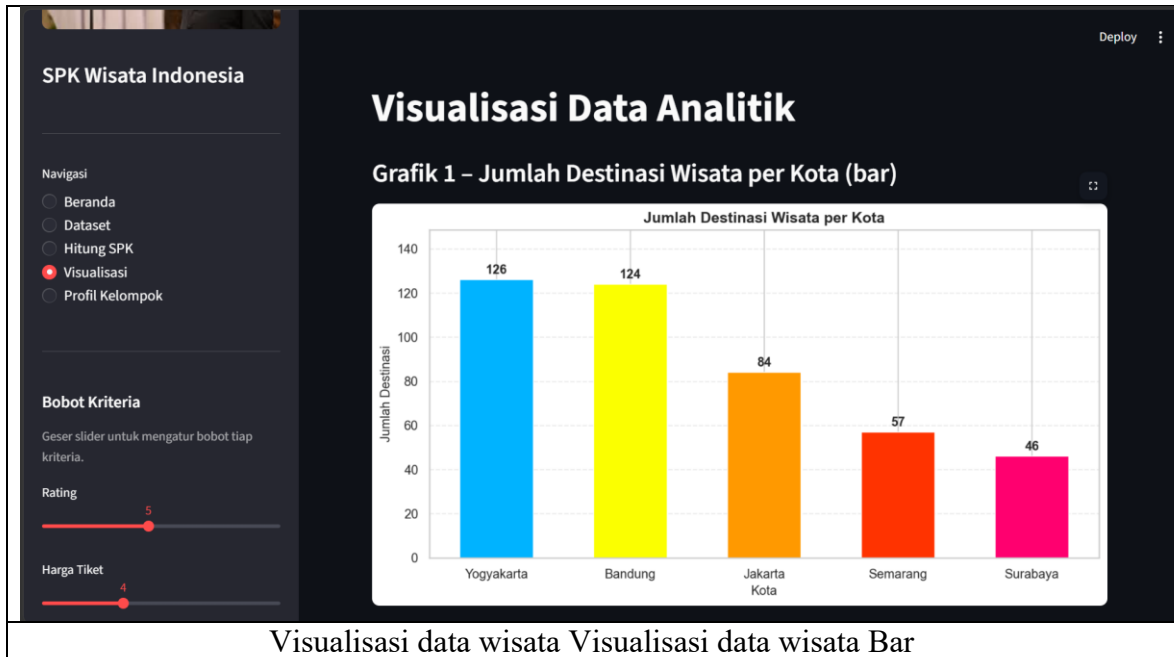
Peringkat	Nama Tempat	Kategori	Kota	Harga (Rp)	Rating	Durasi (menit)	Nilai SAW
1	Taman Budaya Yogyakarta	Budaya	Yogyakarta	0	4.5	210	0.8657
2	Desa Wisata Sungai Code Jogja Kota	Taman Hiburan	Yogyakarta	0	5	60	0.8611
3	Lawangwangi Creative Space	Taman Hiburan	Bandung	0	4.4	120	0.8538
4	Perpustakaan Nasional	Budaya	Jakarta	0	4.7	240	0.8537
5	Kawasan Malioboro	Taman Hiburan	Yogyakarta	0	4.8	60	0.85
6	Gumuk Pasir Parangkusumo	Taman Hiburan	Yogyakarta	0	4.5	90	0.8472
7	Desa Wisata Kelor	Taman Hiburan	Yogyakarta	0	4.4	90	0.8417
8	Bendung Lepen	Taman Hiburan	Yogyakarta	0	4.6	60	0.8389
9	Alun Alun Selatan Yogyakarta	Taman Hiburan	Yogyakarta	0	4.6	60	0.8389
10	Nol Kilometer Jl.Malioboro	Taman Hiburan	Yogyakarta	0	4.7	45	0.8375
11	Alun-Alun Kota Bandung	Taman Hiburan	Bandung	0	4.6	60	0.8371
12	Jembatan Pasupati	Taman Hiburan	Bandung	0	4.6	60	0.8371
13	Taman Badak	Taman Hiburan	Bandung	0	4.5	60	0.8316

Hasil perangkingan destinasi wisata

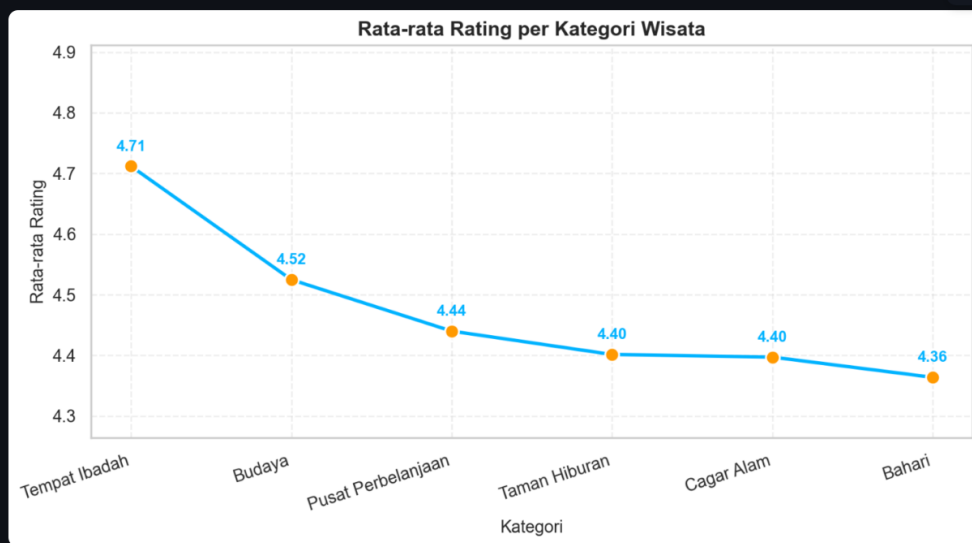
Top 5 Rekomendasi Terbaik

Taman Budaya Yogyakarta	Desa Wisata Sungai Code Jogja Kota	Lawangwangi Creative Space	Perpustakaan Nasional	Kawasan Malioboro Yogyakarta
Yogyakarta	Yogyakarta	Bandung	Jakarta	Taman Hiburan
Budaya	Taman Hiburan	Taman Hiburan	Budaya	4.8
4.5	5.0	4.4	4.7	Rp 0
Rp 0	Rp 0	Rp 0	Rp 0	SAW: 0.8500
SAW: 0.8657	SAW: 0.8611	SAW: 0.8538	SAW: 0.8537	

Tampilan Rekomendai Terbaik dari Taman Hiburan

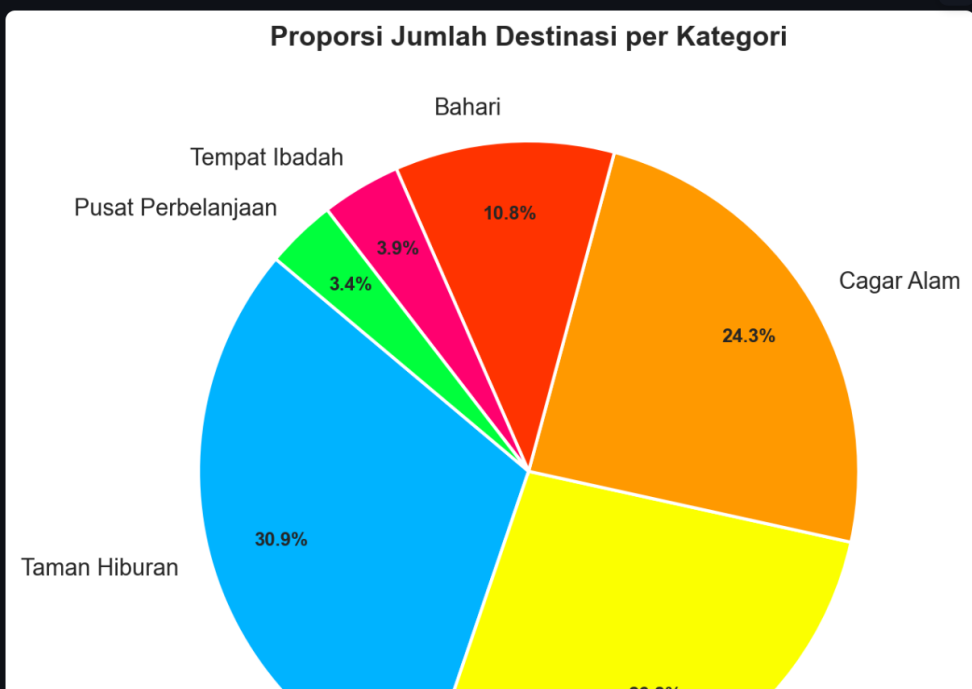


Grafik 3 – Rata-rata Rating per Kategori Wisata (plot)



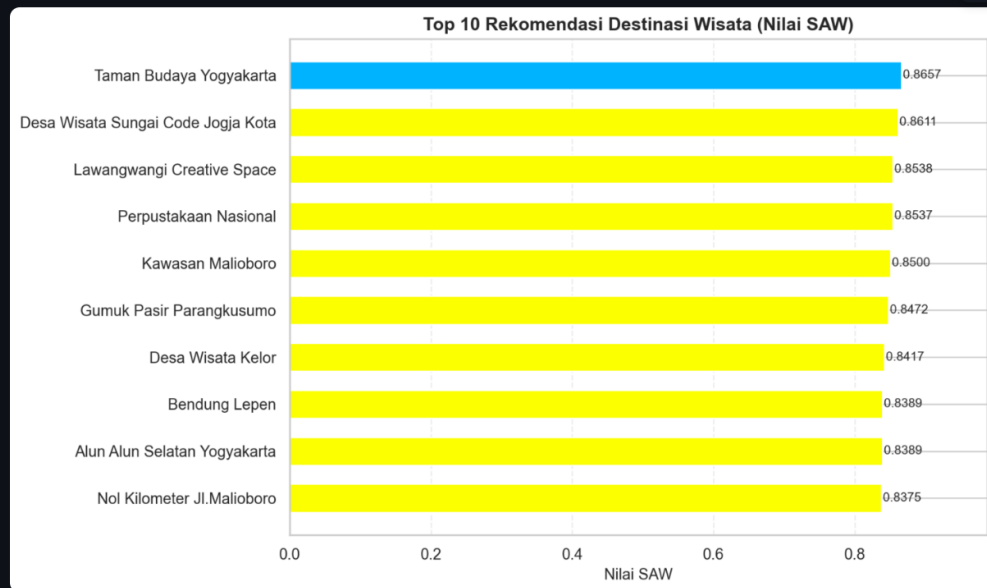
Visualisasi data wisata dalam Plot

Grafik 4 – Proporsi Jumlah Destinasi per Kategori (pie)



Visualisasi data wisata dalam Pie

Grafik 5 – Top 10 Rekomendasi Nilai SAW (bar horizontal)



Visualisasi data wisata ketika sudah di hitung dalam Bar Horizontal

SPK Wisata Indonesia

Navigasi
☐ Beranda
☐ Dataset
☐ Hitung SPK
☐ Visualisasi
☒ Profil Kelompok

Bobot Kriteria
 Geser slider untuk mengatur bobot tiap kriteria.
 Rating
 Harga Tiket

Profil Kelompok

Informasi Proyek

Mata Kuliah	Sistem dan Cerdas Pendukung Keputusan (SCPK)
Tahun Ajaran	2025 / 2026
Metode SPK	Simple Additive Weighting (SAW)
Dataset	Indonesia Tourism Destination (437 baris)
Sumber Data	Kaggle – Indonesia Tourism Destination

Anggota Kelompok

Anggota 1

- Nama : Bisma Putra Pangestu
- NIM : 123240217

Anggota 2

- Nama : Muhammad Syawal Azzami
- NIM : 123240224

Tampilan profil kelompok

BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	2026				
		Minggu				
		1	2	3	4	dst
1	Tema dan Judul					
2	Mencari Data					
3	Pembuatan Program					
4						
dst						

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Bisma Putra Pangestu	123240217	- Program - Laporan - Judul
2	Muhammad Syawal Azzami	123240224	- Cari data - Laporan - Judul

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Berdasarkan hasil proyek akhir yang telah dilakukan, sistem pendukung keputusan rekomendasi tempat wisata di Indonesia berhasil dibangun menggunakan metode *Simple Additive Weighting* (SAW). Sistem mampu melakukan proses perhitungan dan perbandingan destinasi wisata berdasarkan beberapa kriteria yaitu *rating*, harga tiket, durasi kunjungan, preferensi kategori, dan popularitas kota. Dengan adanya sistem ini, pengguna dapat memperoleh rekomendasi tempat wisata terbaik secara lebih cepat dan objektif.

Implementasi sistem menggunakan *Python* dan *Streamlit* juga berhasil memberikan tampilan yang interaktif dan mudah digunakan. Selain dapat melakukan proses perhitungan SAW secara otomatis, sistem juga mampu menampilkan visualisasi data dalam bentuk grafik sehingga informasi menjadi lebih mudah dipahami. Berdasarkan hasil pengujian, metode SAW dapat digunakan dengan baik untuk membantu proses pengambilan keputusan dalam pemilihan destinasi wisata.

4.3 Saran

Sistem yang telah dibuat masih dapat dikembangkan lebih lanjut agar memiliki fitur yang lebih lengkap dan hasil rekomendasi yang lebih optimal. Salah satu pengembangan yang dapat dilakukan adalah menambahkan lebih banyak kriteria seperti jarak lokasi, fasilitas wisata, jumlah pengunjung, serta ulasan pengguna secara *real-time*. Selain itu, *dataset* juga dapat diperbarui secara berkala agar informasi destinasi wisata tetap relevan dan akurat.

Pengembangan lain yang dapat dilakukan yaitu menambahkan fitur *login* pengguna, integrasi dengan *Google Maps*, serta penggunaan metode SPK lain seperti AHP, TOPSIS, atau Fuzzy untuk membandingkan hasil rekomendasi. Tampilan antarmuka sistem juga dapat dibuat lebih menarik dan responsif agar meningkatkan kenyamanan pengguna saat menggunakan aplikasi.

DAFTAR PUSTAKA

- Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Muhammadiyah Sumatera Utara. 2023. "Pengertian Metode SAW Sistem Pendukung Keputusan." Diakses dari <https://fikti.umsu.ac.id/pengertian-metode-saw-sistem-pendukung-keputusan/>. pada 25 Mei 2026.
- TechTarget. 2024. "What is a Decision Support System (DSS)?" Diakses dari <https://www.techtarget.com/searchcio/definition/decision-support-system>. pada 25 Mei 2026.
- Team TrainIT. 2024. "Langkah-Langkah Metode SAW." Diakses dari <https://www.teamtrainit.com/demo/algoritma/saw/langkah.php>. pada 25 Mei 2026.